← Web App | 🎓 Educational Guide | 📖 Manual & CLI | 🧪 Test Report | Download PDF

EN - English

ARCHITECTURE & SYSTEM DESIGN

Cardioid — Software Architecture Specification

Complete architectural description, component interactions, module specifications, PlantUML class diagrams, and API reference.

📌 1. System Overview & Repository Reference

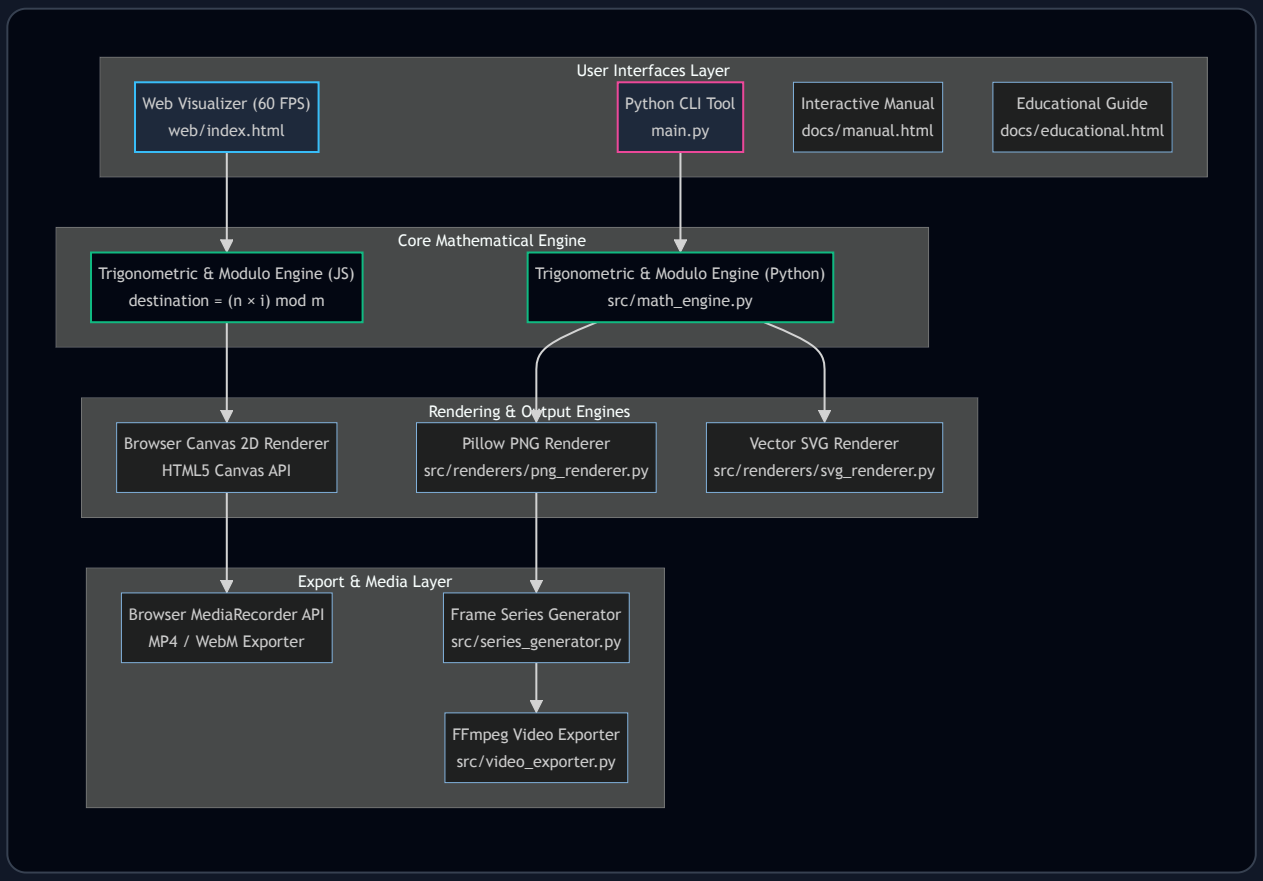
The Cardioid Project is a dual-engine software suite designed for real-time visualization, vector rendering, frame sequence generation, and video compilation of modular multiplication tables geometry.

Attribute	Value / GitHub Location
GitHub Repository	https://github.com/simoscream/Cardioid
Current Version Tag	`v2.0` (Major Release)
Live GitHub Pages URL	https://simoscream.github.io/Cardioid/

Attribute	Value / GitHub Location
Technology Stack	HTML5 Canvas 2D, JavaScript (ES6+), Python 3.12, Pillow, FFmpeg, Pytest, GitHub Actions/Pages

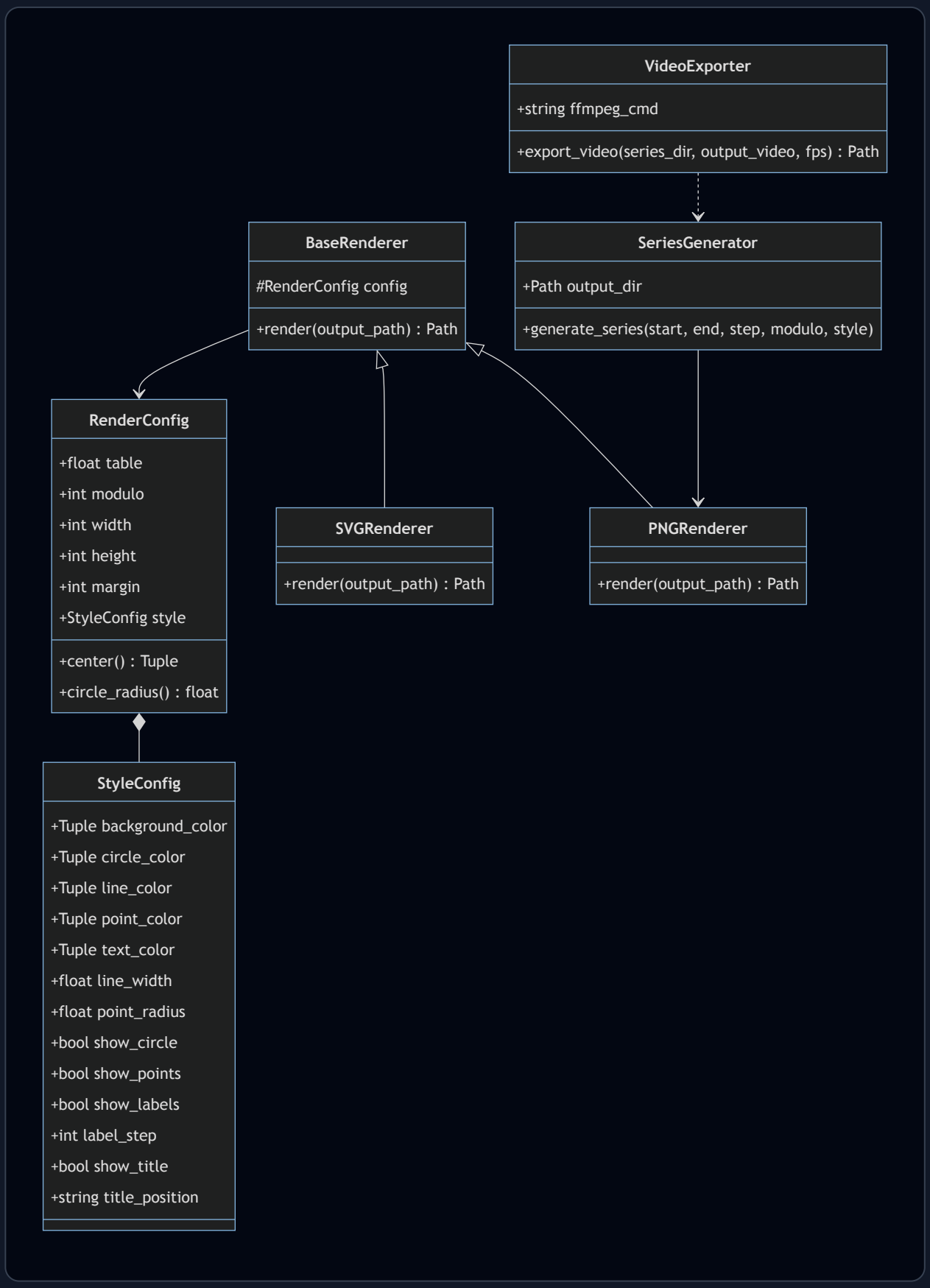
🏗️ 2. High-Level System Architecture

The architecture is split into two independent, complementary execution environments: the Client Web Canvas 2D Engine and the Python CLI Rendering Engine.



📐 2.2 PlantUML & Class Diagram

Object-Oriented structure of configuration dataclasses, abstract renderers, concrete Pillow/SVG implementations, and batch exporters.



Official PlantUML Code Specification:

```
@startuml Cardioid_Class_Diagram

package "Configuration Layer" {
    class StyleConfig {
        + background_color: Tuple[int, int, int]
        + circle_color: Tuple[int, int, int, int]
        + line_color: Tuple[int, int, int, int]
        + point_color: Tuple[int, int, int, int]
        + text_color: Tuple[int, int, int, int]
        + line_width: float
        + point_radius: float
        + show_circle: bool
        + show_points: bool
        + show_labels: bool
        + label_step: int
        + show_title: bool
        + title_position: str
    }

    class RenderConfig {
        + table: float
        + modulo: int
        + width: int
        + height: int
        + margin: int
        + style: StyleConfig
        + center() : Tuple[float, float]
        + circle_radius() : float
    }

    RenderConfig *-- StyleConfig
}

package "Rendering Pipeline" {
    abstract class BaseRenderer {
        # config: RenderConfig
        + {abstract} render(output_path: Path) : Path
    }

    class PNGRenderer {
```

```
        + render(output_path: Path) : Path
    }

    class SVGRenderer {
        + render(output_path: Path) : Path
    }

    BaseRenderer <|-- PNGRenderer
    BaseRenderer <|-- SVGRenderer
    BaseRenderer --> RenderConfig
}

package "Batch & Video Exporters" {
    class SeriesGenerator {
        + output_dir: Path
        + generate_series(start_table: float, end_table: float, step: flo
at, modulo: int, style: StyleConfig) : List[Path]
    }

    class VideoExporter {
        + ffmpeg_cmd: str
        + export_video(series_dir: Path, output_video_path: Path, fps: in
t) : Path
    }

    SeriesGenerator --> PNGRenderer
    VideoExporter ..> SeriesGenerator
}

@enduml
```

🧩 3. Software Modules Specification

web/index.html code

Standalone single-file 60 FPS HTML5 Canvas visualizer. Manages state, real-time animation loop, color pickers, PNG/SVG 1-click downloads, MediaRecorder video capture, and 7-language i18n dictionary.

main.py code

CLI entry point parsing command line arguments (argparse), constructing StyleConfig and RenderConfig, and routing execution to PNG, SVG, Series, or Video exporters.

src/math_engine.py code

Pure mathematical functions: get_point_angle, get_point_coordinates, compute_destination_index, generate_all_points, and generate_chords.

src/config.py code

Dataclass configurations (StyleConfig and RenderConfig) defining colors, line widths, circle radius, title overlay settings, and canvas margins.

src/renderers/ code

Abstract BaseRenderer class with concrete implementations PNGRenderer (Pillow raster image) and SVGRenderer (W3C vector XML string builder).

src/video_exporter.py code

FFmpeg subprocess wrapper compiling frame sequences into H.264 MP4 videos using glob pattern matching.

🔌 4. API Reference Table

Summary of primary internal and browser/system APIs consumed across the codebase:

API Name	Environment / Module	Usage & Purpose
`CanvasRenderingContext2D`	Browser / `web/index.html`	Real-time 60 FPS drawing (arc, lineTo, stroke, fillText).
`MediaRecorder API`	Browser / `web/index.html`	In-browser video recording (video/mp4, video/webm).
`Web Storage API`	Browser / All Pages	Language preference persistence (localStorage.getItem('cardioid_lang')).
`Pillow (PIL.ImageDraw)`	Python / `png_renderer.py`	Raster PNG generation with antialiased lines and alpha channel transparency.
`FFmpeg Subprocess CLI`	System / `video_exporter.py`	Compilation of PNG frame sequences into H.264 MP4 video.